



University of Glasgow

ROBOTICS TEAM DESIGN PROJECT M

ENG-5325

NAO6 ROBOT SOCCER GAME REPORT

SUBMITTED BY

TEAM 9

25.APR.2025

Team Member and Task Allocation Table

Name	Student ID	Project Role	Report Section
Guankai Wang	2996707	Team manager	Introduction Methodology
Zhihong Xu	2997144	Co-leader	Abstract Methodology Appendices
YUWEN Wang	2997095	Developer	Introduction Methodology Appendices
Zhijian Huang	2999277	Developer	Methodology Appendices
Abhishek Jain	2997815	Developer	Conclusions and Future Works

Contents

ABSTRACT	5
INTRODUCTION	6
1. METHODOLOGY	6
1.1 TOOLS AND PLATFORMS:	7
1.2 IMPLEMENTATION OF BASIC FUNCTIONALITIES	10
1.2.1 <i>Forward Kinematics</i>	10
1.2.2 <i>Inverse Kinematics</i>	13
1.2.3 <i>Model Validation and Controller</i>	15
1.2.4 <i>Light kick</i>	16
1.2.5 <i>Powerful kick</i>	16
1.2.6 <i>Tracking Strategy</i>	17
1.2.7 <i>Dribbling Strategy</i>	18
1.3 ROBOT SAFETY CONSIDERATIONS.....	18
1.3.1 <i>Self-Collision Prevention</i>	19
1.3.2 <i>Obstacle Avoidance</i>	19
1.4 COMMUNICATION	21
1.5 INDIVIDUAL ROBOT ALGORITHMS	22
1.5.1 <i>Striker</i>	22
1.5.2 <i>Defender</i>	23
1.5.3 <i>Goalkeeper</i>	25
1.6 TEAM STRATEGY DEVELOPMENT AND TESTING.....	27
1.6.1 <i>Strategy 1 : Three Hybrid Attackers</i>	27
1.6.2 <i>Strategy 2 : One Striker with two Defenders</i>	28
1.6.3 <i>Strategy 3 : One Striker with one Defender and one Flexible Player</i>	28
2 RESULTS AND ANALYSIS	30
2.1 PID CONTROLLER	30
2.2 SAFETY CONSIDERATIONS.....	31
2.3 STRATEGY PERFORMANCE EVALUATION	32
3.TEAM PERFORMANCE ANALYSIS	33

4.CONCLUSIONS AND FURTHER WORK	34
4.1 CONCLUSIONS.....	34
4.1.1 <i>Technical Achievements</i>	34
4.1.2 <i>Teamwork & Collaboration.: 1.</i>	34
4.2 FUTURE WORK	35
5. REFERENCE	36
6. APPENDICES.....	37
6.1 PYTHON CODE FOR LIGHT KICK.....	37
6.2 PYTHON CODE FOR POWERFUL KICK	38
6.3 PYTHON CODE FOR CHECKING WHETHER TO STAND UP OR NOT	38
6.4 PYTHON CODE FOR SAVING BALL	39
6.5 PYTHON CODE FOR APF ALGORITHM	40
6.6 PYTHON CODE FOR RECOVER DEFENSIVELY	41
6.7 THE LINK OF GITHUB.....	41

Abstract

This project shows the design and implementation of a fully autonomous soccer-playing system based on NAO6 robots, inspired by the RoboCup competition framework. Each robot is assigned a specific role, such as like striker, defender, and goalkeeper, and is programmed to perform key soccer behaviors such as kicking, running, dribbling, and intercepting. All actions are executed autonomously without manual intervention, and the robots only rely on sensors such as gyroscopes and inertial units. A communication mechanism is established to facilitate real-time data sharing, with shared information including ball position and team coordination. The system is developed on the Webots simulation platform with the and Python API, using state machines, motion coordination, and balance control. This project presents the work of achieving cooperative, strategy-driven robotic soccer play in a simulated RoboCup environment.

Introduction

The concept of this project is inspired by RoboCup, an international series of competitions centered around robotics and robotic systems. Each team is required to carry out a soccer match using four NAO6 humanoid robots. These robots must be capable of performing realistic soccer player movements and actions, such as kicking, dribbling, running, and intercepting. Each team must assign three specific roles to the robots: striker, defender, and goalkeeper. The striker plays a crucial role in the offensive strategy and is primarily responsible for scoring. To win the game, the striker must adopt a highly aggressive attacking mode. In contrast, the defender is responsible for preventing the ball from entering the team's half of the field and blocking the opponent's attempts to score. The goalkeeper serves as the final line of defense and typically collaborates with the defender to protect the goal area. All robots must operate in fully autonomous mode; manual or remote control is strictly prohibited. Each robot must be able to make decisions regarding its actions and strategies using its own onboard sensors, such as cameras or sonar. Additionally, inter-robot communication is allowed to enable the sharing of information such as ball location and role switching. Details such as field size, match duration, and game rules will follow the official RoboCup regulations.

1. Methodology

The ultimate objective of this project is to simulate a robotic football match that complies with the official RoboCup regulations. Achieving this goal required the development of distinct algorithms for each of the four robots, enabling them to perform role-specific actions within a team-based strategy. Once these individual algorithms were implemented and validated through functional testing, the next step was to design and apply coordinated tactics at the team level, allowing the robots to work together effectively in a match scenario. To ensure a structured and efficient development process, the project was divided into four main stages:

(1) the implementation of basic functionalities such as walking, ball detection, and ball tracking.

(2) safety considerations to maintain robot balance and prevent collisions.

(3) the development of individual robot algorithms, tailored to specific roles like striker, defender, and goalkeeper.

(4) Team Strategy Development and Testing.

The project also focuses on designing and implementing a strategy system for the NAO6 robot in a robot soccer game. To understand the robot's behavior, our team started the research by understanding the kinematics part. Then, we use PID controllers for angle analysis. Kicking actions play a vital role in soccer games. So, we developed a light kick using a custom motion file, and then using a PID controller to generate a powerful kick.

In addition, role specific behaviors and strategies were developed for different roles, which are striker, defender, and goalkeeper. Each role's strategy was responsible for a different strategy. At the end, they were integrated into a coordinated team framework to enhance overall performance during the game.

1.1 Tools and Platforms:

To minimize the unknown obstacles to the progress of the project caused by the difficulty of the programming language as much as possible, we adopted Python as our programming language in this project. Python, as a scripting language, is interpreted, high-level and versatile, making it suitable for rapid development. . It can run directly without using the compiler. This greatly reduces the unnecessary time we spend on compilation. At the same time, the Python language is easy to get started with and is suitable for students without a programming background to develop. The platform we use for robot simulation is Webots, a free and open-source software developed by Cyberbotics that enables 3D robot simulation. It has a physics engine and can simulate robots in the real world very well, such as gravity, friction and object collisions. Then we used PyCharm as the code editing tool. It is an integrated development environment for computer programming developed by the Czech company JetBrains. Its features such as code completion prompts, which are very convenient for developing functions and debugging programs. Finally, we adopted GitHub for code hosting . It is an online software code hosting platform used to expose the code of programs or software and

has been integrated with Git as a version control software. On our project, we can update or roll back the code at will. This enables us to upload the code to GitHub after completing one function with peace of mind and then proceed to develop the next function.

We have integrated these tools and platforms for use in our project, starting with PyCharm and Webots. Because editing code in Webots is a very unfriendly thing, first, it does not support the prompt code function, which is not convenient for coding where the length of code is large, or the code is scattered across various files. Under these circumstances, if you forget the name of a certain function, you will be forced to search for it in other files, which invisibly increases the workload and reduces overall productivity. Secondly, PyCharm can use various plugins to enhance programming efficiency. It even has a Debug mode, which enables code to run from breakpoints. And you can run your program step by step. This is very helpful for identifying program bugs and making timely corrections. To enable PyCharm to interact with Webots, we need to set the correct environment configuration on PyCharm. First, we need to adjust the version of Python to be consistent with that of Webots. In this project, the version of Webots is 3.9, so we need to adjust the version of Python used in PyCharm to 3.9.6. Next, the paths of Webots also need to be configured. As shown in Figure 1, the library file path of Python in Webots, such as Controller, needs to be imported, an executable file path of Webots also needs to be imported. In this way, simulation development can be carried out successfully.

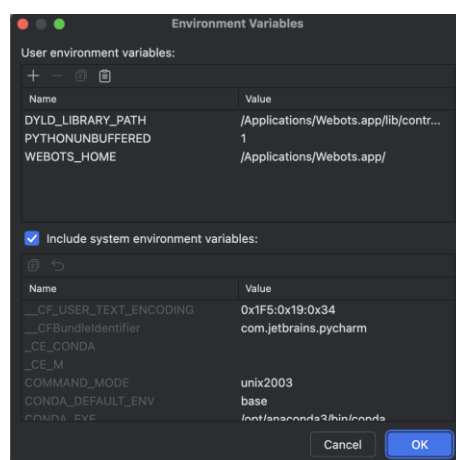


Figure 1. Environment configuration in PyCharm for collaborating with Webots. Including setting the path for the Webots controller libraries.

After we develop a function of this Once a project feature, such as the Powerful Kick, has been implemented and after we conduct tests and confirm that there are no problems, the code is committed and it will be uploaded to GitHub, as shown in Figure 2. The figure illustrates one of the code branches of our project. At the same time, PyCharm would display what the current Git code branch is, whether the current code branch has been updated, and so on. This is thanks to the fact that PyCharm also has been integrated with a Git intergration , which can perform common Git operations such as code upload, update, and download. Moreover, PyCharm can provide a visual interface to the user, which can greatly reduce the psychological burden of operation on developers.

File/Folder	Commit Message	Last Update
.idea	Goalkeeper: 1. Updating teammate detect to avoid collision. 2. Updati...	last month
.vscode	Nao_test_version 1	6 months ago
controllers	Goalkeeper: 1. Updating teammate detect to avoid collision...	last month
libraries	NaoGoalKeeper V1.1	3 months ago
log	Nao_test_version 1.1.2.1	5 months ago
protos	Nao_test_version 1	6 months ago
scripts	Nao_test_version 1.1.2.1	5 months ago
worlds	Add files via upload	2 months ago
.env	Nao_test_version 1	6 months ago
Nao_test.code-workspace	Nao_test_version 1	6 months ago

Figure 2. Project structure on GitHub. Various folders, such as controllers, worlds, and scripts, which are used in the Webots project.

1.2 Implementation of Basic Functionalities

1.2.1 Forward Kinematics

For humanoid robots, limb movements such as those of the elbow, legs, and feet are achieved by actuating the servo motors located at each joint. By simultaneously controlling the rotation angles and rotation speeds of different motors, the robot is able to perform complete actions or generate walking gaits.

From the perspective of implementing a simulated match, our goal is to make the robot perform as expected. Essentially, this is done by sending target rotation angles and velocities to the motors at each joint, so that the end effector (e.g., hand or foot) reaches the desired position. This principle directly corresponds to the concepts of Forward Kinematics (FK) and Inverse Kinematics (IK) in robotics.

Forward kinematics refers to the process of determining the position of the end effector when the rotation angles of all joint motors are known. Mathematically, we typically use the Denavit–Hartenberg (DH) convention to describe the geometric relationships between joints. By applying homogeneous transformation matrices and multiplying them sequentially, we construct a transformation chain that ultimately provides the position and orientation of the end effector.

We reviewed relevant literature [1] and used the following Denavit–Hartenberg (DH) parameter table to establish the mathematical model.

Table 1. The DH parameters for the head. Source: [1]

Joint frame	a	α	d	θ
Base		$A(0, 0, \text{Neck Offset } Z)$		
Head Yaw (Pos 1)	0	0	0	θ_1
Head Pitch (Pos 2)	0	$-\pi/2$	0	$\theta_2 - \pi/2$
Rotation		$R_x(\pi/2)R_y(\pi/2)$		
T Camera (Top)		$A(\text{T Camera } X, 0, \text{T Camera } Z)$		
B Camera (Bottom)		$A(\text{B Camera } X, 0, \text{B Camera } Z)$		
T Camera $X = 53.9$ mm, T Camera $Z = 67.9$ mm, B Camera $X = 48.8$ mm, B Camera $Z = 23.8$ mm				

Table 2. The DH parameters for the left arm. Source:[1]

Joint frame	a	α	d	θ
Base	$A(0, \text{Shoulder Offset } Y + \text{Elbow Offset } Y, \text{Shoulder Offset } Z)$			
L Shoulder Pitch (Pos 1)	0	$-\pi/2$	0	θ_1
L Shoulder Roll (Pos 2)	0	$\pi/2$	0	$\theta_2 - \pi/2$
L Elbow Yaw (Pos 3)	0	$-\pi/2$	Upper arm length	θ_3
L Elbow Roll (Pos 4)	0	$\pi/2$	0	θ_4
Rotation	$R_z(\pi/2)$			
End effector	$A(\text{Hand Offset } X + \text{Lower Arm Length}, 0, 0)$			

Table 3. The DH parameters for the right arm. Source: [1]

Frame (joint)	a	A	d	θ_1
Base	$A(0, -\text{Shoulder Offset } Y - \text{Elbow Offset } Y, \text{Shoulder Offset } Z)$			
R Shoulder Pitch (Pos 1)	0	$-\pi/2$	0	θ_2
R Shoulder Roll (Pos 2)	0	$\pi/2$	0	$\theta_2 + \pi/2$
R Elbow Yaw (Pos 3)	0	$-\pi/2$	$-\text{Upper arm length}$	θ_3
R Elbow Roll (Pos 4)	0	$\pi/2$	0	θ_4
Rotation	$R_z(\pi/2)$			
End effector	$A(-\text{Hand Offset } X - \text{Lower Arm Length}, 0, 0)$			
Rotation fix	$R_z(-\pi)$			

Table 4. The DH parameters for the left leg. Source: [1]

Frame (joint)	a	α	d	θ
Base	$A(0, \text{Hip Offset } Y, -\text{Hip Offset } Z)$			
L Hip Yaw Pitch (Pos 1)	0	$-3\pi/4$	0	$\theta_1 - \pi/2$
L Hip Roll (Pos 2)	0	$-\pi/2$	0	$\theta_2 + \pi/4$
L Hip Pitch (Pos 3)	0	$\pi/2$	0	θ_3
L Knee Pitch (Pos 4)	$-\text{Thigh length}$	0	0	θ_4
L Ankle Pitch(Pos 5)	$-\text{Thigh length}$	0	0	θ_5
L Ankle Roll (Pos 6)	0	$-\pi/2$	0	θ_6
Rotation	$R_z(\pi) R_y(-\pi/2)$			
End effector	$A(0, 0, -\text{Foot Height})$			

Table 5. The DH parameters for the right leg. Source: [1]

Frame (joint)	a	α	d	θ
Base	$A(0, -\text{Hip Offset } Y, -\text{Hip Offset } Z)$			
R Hip Yaw Pitch (Pos 1)	0	$-\pi/4$	0	$\theta_1 - \pi/2$
R Hip Roll (Pos 2)	0	$-\pi/2$	0	$\theta_2 - \pi/4$
R Hip Pitch (Pos 3)	0	$\pi/2$	0	θ_3
R Knee Pitch (Pos 4)	–Thigh length	0	0	θ_4
R Ankle Pitch (Pos 5)	–Thigh length	0	0	θ_5
R Ankle Roll (Pos 6)	0	$-\pi/2$	0	θ_6
Rotation	$R_z(\pi)R_y(-\pi/2)$			
End effector	$A(0, 0, -\text{Foot Height})$			

The final transformation matrices were calculated as follows, with the **torso** defined as the origin of the coordinate frame:

Head:

$$T_{Base}^{End} = A_{Base}^0 T_0^{Pos1} T_{Pos1}^{Pos2} R_x\left(\frac{\pi}{2}\right) A_{Pos2}^{End} \quad 1.2.1 - 1$$

Left Arm:

$$T_{Base}^{End} = A_{Base}^0 T_0^{Pos1} T_{Pos1}^{Pos2} T_{Pos2}^{Pos3} T_{Pos3}^{Pos4} R_z\left(\frac{\pi}{2}\right) A_{Pos4}^{End} \quad 1.2.1 - 2$$

Right Arm:

$$T_{Base}^{End} = A_{Base}^0 T_0^{Pos1} T_{Pos1}^{Pos2} T_{Pos2}^{Pos3} T_{Pos3}^{Pos4} R_z\left(\frac{\pi}{2}\right) A_{Pos4}^{End} R_z(-\pi) \quad 1.2.1 - 3$$

Left Leg :

$$T_{Base}^{End} = A_{Base}^0 T_0^{Pos1} T_{Pos1}^{Pos2} T_{Pos2}^{Pos3} T_{Pos3}^{Pos5} T_{Pos5}^{Pos6} R_z(\pi) R_y\left(-\frac{\pi}{2}\right) A_{Pos6}^{End} \quad 1.2.1 - 4$$

Right Leg:

$$T_{Base}^{End} = A_{Base}^0 T_0^{Pos1} T_{Pos1}^{Pos2} T_{Pos2}^{Pos3} T_{Pos3}^{Pos5} T_{Pos5}^{Pos6} R_z(\pi) R_y\left(-\frac{\pi}{2}\right) A_{Pos6}^{End} \quad 1.2.1 - 5$$

Finally , we generate a Python code to calculate the specific value for us. In this case, the system will calculate how each part should move to reach the position automatically after we set a specific position for each angle. This can greatly simplify

the motion planning process.

1.2.2 Inverse Kinematics

Relying solely on forward kinematics to compute the position of the end effector from given joint angles is far from sufficient and does not fully meet the expectations of our project. To improve the completeness of our kinematic modelling, it was necessary to also establish an inverse kinematics model for the robot.

In contrast to forward kinematics, inverse kinematics begins with the known position of the end effector and works backward to calculate the required joint angles. This process enables mutual verification between the forward and inverse kinematics models, ensuring that the calculated joint angles can accurately produce the desired end-effector positions, and vice versa. Through an extensive review of related literature, we identified and adopted the inverse kinematics equations specifically tailored for the NAO humanoid robot.

$$\theta_2 = \text{asin} \left(\frac{-p_z + l_3}{\sqrt{l_1^2 + l_2^2}} \right) - \text{atan} \left(\frac{l_1}{l_2} \right) + \frac{\pi}{2}$$

$$\theta_2 = \pi - \text{asin} \left(\frac{-p_z + l_3}{\sqrt{l_1^2 + l_2^2}} \right) - \text{atan} \left(\frac{l_1}{l_2} \right) + \frac{\pi}{2}$$

$$\theta_1 = \pm \text{acos} \left(\frac{p_x}{l_2 \cos \left(\theta_2 - \frac{\pi}{2} \right) - l_1 \sin \left(\theta_2 - \frac{\pi}{2} \right)} \right)$$

provided a target translation (p_x, p_y, p_z) , or

$$\theta_1 = a_z \quad \theta_2 = a_y$$

provided a target orientation (a_x, a_y, a_z)

Figure 3. The inverse kinematics equations for the head. Source:[2]

$$\begin{aligned}
\theta_4 &= - \left(\pi - \text{acos} \left(\frac{l_3^2 + l_4^2 - \|\bar{s} - \bar{p}\|_2}{2l_3l_4} \right) \right) \\
\theta_2 &= \pm \text{acos} \left(\frac{\mathbf{T}_{(2,4)} - l_1 - \left(\frac{l_4 \sin \theta_4 \mathbf{T}_{(2,2)}}{\cos \theta_4} \right)}{l_3 + l_4 \cos \theta_4 + l_4 \frac{\sin^2 \theta_4}{\cos \theta_4}} \right) + \frac{\pi}{2} \\
\theta_3 &= \text{asin} \left(\frac{\mathbf{T}_{(2,3)}}{\sin \left(\theta_2 - \frac{\pi}{2} \right)} \right) \quad \theta_3 = \pi - \text{asin} \left(\frac{\mathbf{T}_{(2,3)}}{\sin \left(\theta_2 - \frac{\pi}{2} \right)} \right) \\
\theta_1 &= \begin{cases} \pm \text{acos} \left(\frac{\mathbf{T}_{(3,3)} + \frac{\mathbf{T}_{(1,3)} \sin \theta_3 \cos \left(\theta_2 - \frac{\pi}{2} \right)}{\cos \theta_3}}{\cos \theta_3 + \frac{\cos^2 \left(\theta_2 - \frac{\pi}{2} \right) \sin^2 \theta_3}{\cos \theta_3}} \right) & \text{if } \theta_3 \neq \frac{\pi}{2} \\ \pm \text{acos} \left(\frac{\mathbf{T}_{(1,3)}}{\cos \left(\theta_2 - \frac{\pi}{2} \right) \sin \theta_3} \right) & \text{if } \theta_3 = \frac{\pi}{2} \end{cases}
\end{aligned}$$

Figure 4. The inverse kinematics equations for the left arm. Source:[2]

$$\begin{aligned}
\mathbf{T}' &= \left(\mathbf{R}_x \left(\frac{\pi}{4} \right) \left(\left(\mathbf{A}_{\text{Base}}^0 \right)^{-1} \mathbf{T} \left(\mathbf{A}_6^{\text{End}} \right)^{-1} \right) \right)^{-1} \\
\theta_4 &= \pm \left(\pi - \text{acos} \left(\frac{l_1^2 + l_2^2 - \|\bar{0} - \bar{p}\|_2}{2l_1l_2} \right) \right) \\
\theta_6 &= \begin{cases} \text{atan} \left(\frac{\mathbf{T}'_{(2,4)}}{\mathbf{T}'_{(3,4)}} \right) & \text{if } (l_2 \cos \theta_5 + l_1 \cos (\theta_4 + \theta_5)) \neq 0 \\ \text{undefined} & \text{if } (l_2 \cos \theta_5 + l_1 \cos (\theta_4 + \theta_5)) = 0 \end{cases} \\
\mathbf{T}'' &= \left((\mathbf{T}')^{-1} \left(\mathbf{T}_5^6 \mathbf{R}_z(\pi) \mathbf{R}_y \left(-\frac{\pi}{2} \right) \right)^{-1} \right)^{-1} \\
\theta_5 &= \text{asin} \left(- \frac{\mathbf{T}''_{(2,4)} (l_2 + l_1 \cos \theta_4) + l_1 \mathbf{T}''_{(1,4)} \sin \theta_4}{l_1^2 \sin^2 \theta_4 + (l_2 + l_1 \cos \theta_4)^2} \right) \\
\theta_5 &= \pi - \text{asin} \left(- \frac{\mathbf{T}''_{(2,4)} (l_2 + l_1 \cos \theta_4) + l_1 \mathbf{T}''_{(1,4)} \sin \theta_4}{l_1^2 \sin^2 \theta_4 + (l_2 + l_1 \cos \theta_4)^2} \right) \\
\mathbf{T}''' &= (\mathbf{T}'')^{-1} (\mathbf{T}_3^4 \mathbf{T}_4^5)^{-1} \\
\theta_2 &= \pm \text{acos} \left(\mathbf{T}'''_{(2,3)} \right) - \frac{\pi}{4} \\
\theta_3 &= \text{asin} \left(\frac{\mathbf{T}'''_{(2,2)}}{\sin \left(\theta_2 + \frac{\pi}{4} \right)} \right) \quad \theta_3 = \pi - \text{asin} \left(\frac{\mathbf{T}'''_{(2,2)}}{\sin \left(\theta_2 + \frac{\pi}{4} \right)} \right) \\
\theta_1 &= \pm \text{acos} \left(\frac{\mathbf{T}'''_{(1,3)}}{\sin \left(\theta_2 + \frac{\pi}{4} \right)} \right) + \frac{\pi}{2}
\end{aligned}$$

Figure 5. The inverse kinematics equations for the left leg. Source:[2]

As the [2] shows, the right arm chain is almost identical to the left arm chain. The only difference is in the forward kinematics solution, since there is one more rotation at the end of the chain (Eq. 4). This last rotation can be easily removed following the fourth step of our methodology, by multiplying both sides of the equations from the right with $(R_z(-\pi))^{-1}$. The right leg chain is identical to the left leg chain. The only difference is the DH parameter α of the first joint. Thus, we must multiply instead with $R_x(-\frac{\pi}{4})$.

1.2.3 Model Validation and Controller

After completing the kinematic model, we conducted mutual validation using both forward and inverse kinematics and found that the discrepancy between theoretical and computed values was minimal and could be considered negligible.

We then applied the kinematic model in the Webots simulation environment to control the robot's movement. However, we observed that the robot was only able to move forward in very small steps and often lost balance and fell.

Upon further investigation, we found that directly setting the target rotation angles for the motors resulted in significant discrepancies between the actual and target angles during simulation. In some cases, the actual joint angles even exceeded the intended values.

This mismatch led to poor motion stability, making the robot's movement unreliable and unable to meet the task requirements effectively.

To deal with this issue, we implemented a PID controller. PID stands for Proportional, Integral, and Derivative, and is a classic feedback control algorithm widely used in control systems. A PID controller is commonly employed to adjust system behavior based on real-time error correction. The P (Proportional) term adjusts the control signal in proportion to the current error. The I (Integral) term accumulates past errors to eliminate long-term bias. The D (Derivative) term predicts future errors by considering the rate of change of the error.

In this project, the PID controller was applied to reduce the error between the

actual and desired motor angles. By minimizing the angle error at the motor level, the positional error of the end-effector was also reduced. This contributed to greater stability and consistency in the robot's motion, which is crucial for executing complex behaviors like walking, kicking, and turning.

1.2.4 Light kick

At first, we used Python code to implement a light kick. However, in order to improve the kick stability and the ability to control the kicking motion, we decided to convert the code into a motion file. The reasons for this step were to reduce control complexity, improve motion smoothness, and ensure consistency. Motion files can send the position commands directly, resulting in smoother transitions compared to sending raw position commands directly. Then, we apply the same approach to generate the light kick for both legs, which can increase the flexibility of our player. It allows us to determine which leg the robot should use for the kick in different situations.

1.2.5 Powerful kick

After implementing a light kick, our team aimed to create a more aggressive kicking motion. So, we decided to generate a powerful kick. At first, we initially use the same approach as the light kick, setting the angle and the velocity directly.

However, after multiple tests and simulations, we found that the robot often kicked the ground or lost balance during the leg-lifting phase. These issues were mainly caused by an unstable center of gravity and excessive motor acceleration. This indicated that we could not apply the same logic and methodology used in the light kick to develop a powerful kick motion file.

To ensure stability during the powerful kick process, we once again introduced the PID controller. We implemented the PID control logic in Python and applied it specifically to the powerful kick function. After coding and integrating this controller, we conducted extensive testing again, and the results were highly successful.

By using the PID controller to manage the ankle and hip joint movements, we were able to continuously monitor the error between the target and actual joint angles, thus avoiding over-rotation and loss of balance. This approach resulted in a much more

stable and reliable kicking motion.

1.2.6 Tracking Strategy

We used the form of computational vectors to complete the position calculation of the ball and the robot. First, after obtaining the positions of the robot and the ball, calculate the difference between the X-axis and Y-axis of the ball and the robot, set as dx and dy. Then, calculate the Euclidean distance between the robot and the ball based on dx and dy. Next, take dx and dy as vectors and standardize them by using the distances that you just obtained to get the unit vector. Then, based on the obtained direction vector of the front of the robot, it is also normalized, and the unit direction vectors can be obtained. Then, the obtained unit vector and unit direction vector are subjected to the dot product. Therefore, we can get the radian value between the front of the robot and the ball. Then, using the function of converting radians to angles, it is converted to the Angle value. Finally, by performing the cross product on the unit vector and the unit direction vector, it can determine whether the ball is on the left or right of the robot based on the positive or negative result of the cross product. When the result of the cross product is positive, the ball is on the right side of the robot. When the result of the cross product is negative, the ball is on the left side of the robot. The specific Python code is shown as Figure 6:

```
def angleCalculator(self, football_position, robot_position, orientation = None): 17 usages ± xzh
    dx = football_position[0] - robot_position[0]
    dy = football_position[1] - robot_position[1]
    target_magnitude = np.sqrt(dx ** 2 + dy ** 2)
    target_vector_normalized = [dx / target_magnitude, dy / target_magnitude]
    if orientation is not None:
        front_magnitude = np.sqrt(orientation[0] ** 2 + orientation[3] ** 2)
        front_vector_normalized = [orientation[0] / front_magnitude, orientation[3] / front_magnitude]
        dot_product = sum(f * t for f, t in zip(front_vector_normalized, target_vector_normalized))
        angle = np.rad2deg(np.arccos(dot_product))
        cross_product = front_vector_normalized[0] * target_vector_normalized[1] - \
            front_vector_normalized[1] * target_vector_normalized[0]
        # print(f"cross_product: {cross_product}")
        if cross_product < 0:
            angle = -angle
    else:
        angle = np.rad2deg(np.arctan2(dy, dx))
    distance = np.sqrt(dx ** 2 + dy ** 2)
    return angle, distance
```

Figure 6. The code represents the algorithm for calculating the angle and the direction between the football and the robot.

This algorithm can not only calculate the angle and direction between the football and the robot but also be used to calculate the Angle and direction between the robot and the goal, as well as among robots themselves. Generally, this is the basic algorithm of the entire project, and all functional implementations are developed based on this algorithm.

1.2.7 Dribbling Strategy

Initially, our first strategy for dribbling is to continuously dribble while walking. Discovering that due to the shape of the robot's foot and the position of the ball, the ball would not move in a straight line. It will go off the track and did not follow the path that we expected. Applying our own dribbling strategy to deal with this issue.

First, the robot will detect the position of the ball and walk toward it. The robot will continuously check the distance between itself and the ball. Setting a threshold for the distance, once the value of the distance falls below the threshold, which means the robot is close enough. Then, the robot performs a kick. After each kick, it will check the distance again and decide whether to approach the ball or do the other kick. This approach is easier to control. It can make sure that the ball will always follow the correct path, reducing the chance of the ball rolling in unpredictable directions.

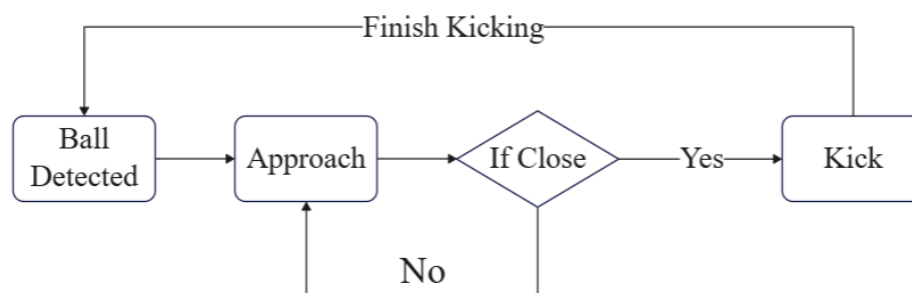


Figure 7. Flow chart for the dribbling strategy.

1.3 Robot Safety Considerations

The second phase of the project was originally planned to apply the strategies and tactics validated in the simulation to physical hardware experiments. Although we ultimately did not proceed to this hardware phase, we still approached the task from a real-world perspective and carefully considered hardware-level safety protection for the

robot.

The safety considerations for hardware protection were mainly focused on two aspects: Self-Collision Prevention and Obstacle Avoidance.

1.3.1 Self-Collision Prevention

We implemented a function to determine whether the robot is currently balanced. In this function, it is determined whether the robot is in a balanced state by judging whether the values of the X-axis and Y-axis of the gyroscopes are zero. Before executing each command, the system first checks whether the current motion file has finished playing and whether the robot is balanced. Only if both conditions are met does the robot proceed with the next command. This approach significantly reduces the likelihood of robot falling due to motion file conflicts or unresolved motor angles or motion moving too fast. The corresponding implementation code is illustrated in Figure 8.

```
def is_balanced(self):  
    vel = self.gyro.getValues()  
    all_in_balance = np.isclose(np.float64(vel[0]),0.0, atol=0.1) and np.isclose(np.float64(vel[1]),0.0, atol=0.1)  
    return all_in_balance
```

Figure 8. The code checks whether the robot is balanced.

1.3.2 Obstacle Avoidance

Through multiple experiments, we observed that collisions not only occurred between our robots and the opponents, but also frequently happened among our own robots. These collisions significantly reduced the efficiency of both offensive and defensive plays.

To address this issue, we implemented an obstacle avoidance algorithm based on the Artificial Potential Field (APF). In this approach, the entire field is considered as a potential field, where all other robots are treated as dynamic obstacles. Each of these obstacles generates a repulsive force in the surrounding space, while the target position generates an attractive force. The resultant force, which is the sum of repulsive and attractive forces, determines the robot's next movement direction.

The main advantages of the APF algorithm are its simplicity, intuitiveness, and

strong real-time performance. However, a known limitation is that it may get trapped in a local minimum, where the robot is stuck in a position with no net force but far from the goal.

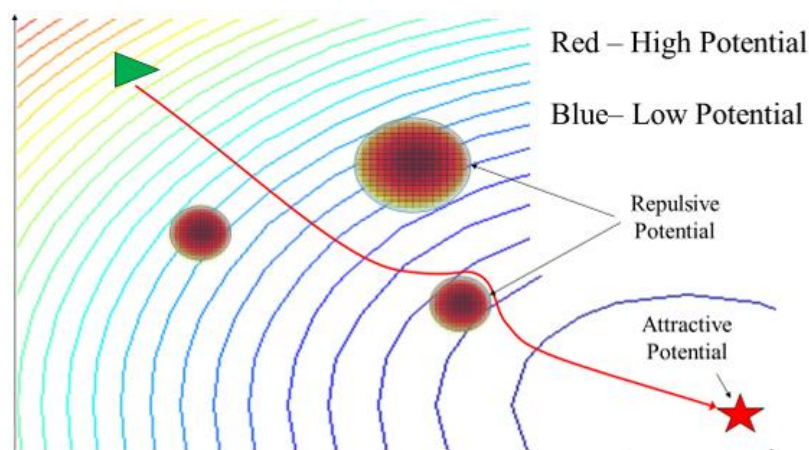


Figure 9. Artificial Potential Field algorithm (from the AVGS lecture) [3]

Artificial Potential Field (APF) method simulates a virtual potential field which the destination is placed at the lowest potential point. Meanwhile, there are an uncertain number of obstacles scattered within the field. Although the specific formulations may vary, the general principle is that the goal exerts an attractive force that pulls the robot toward it, while obstacles exert repulsive forces that push the robot away from them.

In our implementation of the APF algorithm, we adopted the following set of equations:

$$F_{obs} = \sum \alpha * \log \left(1 + \frac{R - d}{R} \right) (d < R) \quad 1.3.2 - 1$$

$$F_{destination} = 1 \quad 1.3.2 - 2$$

$$F_{total} = F_{destination} + F_{obs} \quad 1.3.2 - 3$$

After normalization, we will obtain a direction vector pointing toward the target. This direction vector, together with the robot's current orientation, is then passed into a calculation function. Through a series of simple vector operations, we can determine the relative angle of the target with respect to the robot's local coordinate frame, which is then used for navigation.

1.4 Communication

In our Webots simulation, we set one of the robots to Supervisor mode, like the blue one Striker, which can get the position and orientation of every object on field, and then we communicate between the robots through the communication mechanism in Webots. At this point, the Goalkeeper of blue side packs the position and orientation of each robot and football in JSON format and broadcasts it to each robot through Emitter module. After other robots receive the broadcast JSON format communication packets by using Receiver module, they will parse them, extract the required position information, and the pass it to the decision module for logical judgement and execution of the robot's action behavior. As shown in Figure 10 and Figure 11:

```
def refresh_position(self): 1usage 1 xzh*
    for i in self.__node_sheet.keys():
        self.__node_position[i] = self.__node_sheet[i].getPosition()
        self.__node_orientation[i] = self.__node_sheet[i].getOrientation()

    self.__shared_info = {
        "striker_red": {"position": self.__node_position["striker_red"], "orientation": self.__node_orientation["striker_red"]},
        # "defender_right": {"position": nodePosition["defender_right"], "orientation": nodeOrientation["defender_right"], "isholdingball":
        "striker_blue": {"position": self.__node_position["striker_blue"], "orientation": self.__node_orientation["striker_blue"]},
        "football": {"position": self.__node_position["football"], "orientation": self.__node_orientation["football"]},
        "goalkeeper_red": {"position": self.__node_position["goalkeeper_red"],
                           "orientation": self.__node_orientation["goalkeeper_red"]},
        "goalkeeper_blue": {"position": self.__node_position["goalkeeper_blue"],
                             "orientation": self.__node_orientation["goalkeeper_blue"]},
        "RedTeam_DefenderLeft": {"position": self.__node_position["RedTeam_DefenderLeft"],
                                 "orientation": self.__node_orientation["RedTeam_DefenderLeft"]},
        "RedTeam_DefenderRight": {"position": self.__node_position["RedTeam_DefenderRight"],
                                   "orientation": self.__node_orientation["RedTeam_DefenderRight"]},
        "BlueTeam_DefenderLeft": {"position": self.__node_position["BlueTeam_DefenderLeft"],
                                   "orientation": self.__node_orientation["BlueTeam_DefenderLeft"]},
        "BlueTeam_DefenderRight": {"position": self.__node_position["BlueTeam_DefenderRight"],
                                    "orientation": self.__node_orientation["BlueTeam_DefenderRight"]},
        "stadiumgoal_red": {"position": self.__node_position["stadiumgoal_red"],
                             "orientation": self.__node_orientation["stadiumgoal_red"]},
        "stadiumgoal_blue": {"position": self.__node_position["stadiumgoal_blue"],
                              "orientation": self.__node_orientation["stadiumgoal_blue"]},
    }

    # print(self.__shared_info)
    temp = json.dumps(self.__shared_info).encode("utf-8")
    self.emitter.send(temp)
```

Figure 10. The code for the emitter.

```

def position_refresh(self): 2 usages  xzh
    if self.receiver.getQueueLength() > 0:
        data = self.receiver.getString()
        shared_info = json.loads(data)
        robot_position = robot_orientation = mate_defender_left_position = mate_defender_right_position\
        = mate_striker_position\
        = mate_defender_left_orientation\
        = mate_defender_right_orientation\
        = mate_striker_orientation \
        = mate_stadium_position \
        = mate_stadium_orientation = None
        if self.__goalkeeper_name == GOAL_KEEPER_ROLE.REDTEAM_GOALKEEPER.value:...
        elif self.__goalkeeper_name == GOAL_KEEPER_ROLE.BLUETEAM_GOALKEEPER.value:...
        else:...

        football_position = np.float64(shared_info["football"]["position"])

        self.receiver.nextPacket()
        self.__robot_position = robot_position
        self.__robot_orientation = robot_orientation
        self.__football_position = football_position
        self.__mate_defender_left_position = mate_defender_left_position
        self.__mate_defender_right_position = mate_defender_right_position
        self.__mate_striker_position = mate_striker_position
        self.__mate_defender_left_orientation = mate_defender_left_orientation
        self.__mate_defender_right_orientation = mate_defender_right_orientation
        self.__mate_striker_orientation = mate_striker_orientation
        self.__mate_stadium_position = mate_stadium_position
        self.__mate_stadium_orientation = mate_stadium_orientation
        self.__ball_position_history.append(self.__football_position)
        if len(self.__ball_position_history) > self.__max_position_history:...

        self.__acc_history.append(self.accelerometer.getValues())
        if len(self.__acc_history) > self.__max_acc_history:...
        return True
    # return robot_position, robot_orientation, football_position
    else:...

```

Figure 11. The code for the receiver.

1.5 Individual robot algorithms

1.5.1 Striker

In football matches, the striker is the primary scorer. According to the official RoboCup competition rules, the initial positions of robots from both teams must be symmetrical, meaning that the striker from each side starts at an equal distance from the ball.

Through extensive testing, we observed that the robot's forward speed is significantly faster than its turning or lateral movement. Therefore, at the start of the match, we designed the striker to wait in place and set a threshold range. When the

opponent, while dribbling the ball, enters this threshold, our striker will trigger a light kick to knock the ball away and immediately move forward. Since the ball is kicked away, the opponent must rotate or shift laterally to face the ball again, while our striker can move directly forward. This speed advantage allows the striker to complete the interception and look for an opportunity to break through. If a breakthrough is not feasible, the striker will call the light kick function again to pass the ball back to a teammate.

This interception behavior is only triggered at the beginning of the match. Once it has been executed, the striker switches to a dribbling strategy that we have designed. Under this strategy, whenever the striker enters the ball's threshold range, it does not immediately kick. Instead, it first performs goal direction evaluation. When the striker reaches the ball, it calculates the direction of the opponent's goal and adjusts its orientation accordingly. Only after aligning with the goal does it perform a kick.

Furthermore, we established a goal-centered threshold zone. If the striker is in possession of the ball within this zone, it will begin to detect the position of the opponent's goalkeeper. If the goalkeeper is positioned toward the left side of the goal (from the striker's perspective), the striker will perform a left-footed kick (shooting to the right). If the goalkeeper is on the right, the striker will kick with the right foot (shooting to the left).

1.5.2 Defender

In the defender's behavior logic, a critical concept is the interception point. In the early version of the algorithm, all robots would directly move toward the ball, effectively treating the ball itself as the intercept point. We improved this approach by designing a new interception strategy based on the core principle of minimizing interception time.

If a defender positions itself on the line connecting the ball and the goal, it can effectively block the attacking path. Therefore, it is best to find the fastest route for the defender to reach any point on this line.

From a geometric perspective, the shortest path from the defender to the ball-goal line is the perpendicular line drawn from the defender to that segment. Based on this

principle, we calculate the intercept point using vector operations.

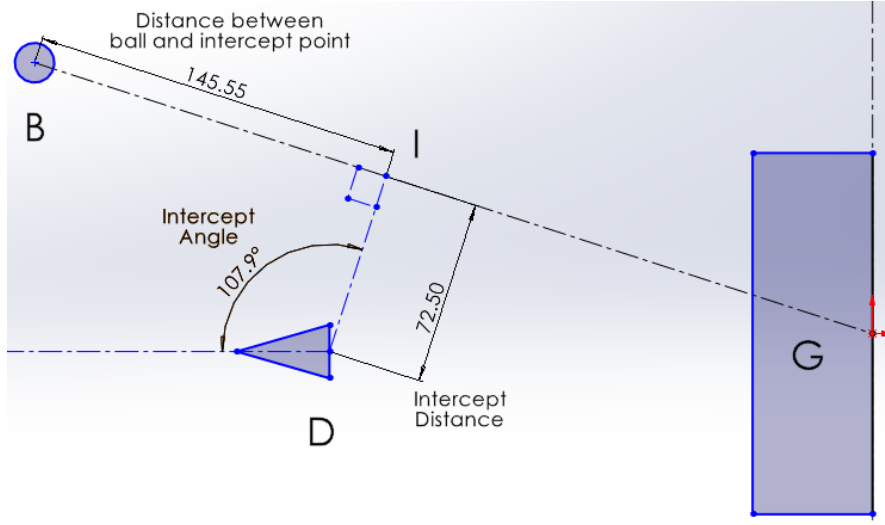


Figure 12. The calculation of intercept point.

Let P denote a position vector, with subscripts representing the corresponding points in the figure12: B stands for Ball. D stands for Defender. I stands for Intercept Point. G stands for Goal. The coordinate calculation process for the intercept point is as follows:

$$\overrightarrow{BG} = P_G - P_B; \overrightarrow{DG} = P_G - P_D \quad 1.5.2 - 1$$

$$\overrightarrow{IG} = \frac{\overrightarrow{BG} * \overrightarrow{DG}}{|\overrightarrow{BG}|^2} \quad 1.5.2 - 2$$

$$\overrightarrow{DI} = \overrightarrow{DG} - \overrightarrow{IG} \quad 1.5.3 - 3$$

$$P_I = P_D + \overrightarrow{DI} \quad 1.5.3 - 4$$

The above method describes the basic calculation of the intercept point. However, we must also consider scenarios where the ball has already passed the defender. If no countermeasure is taken in such cases, the defender will attempt an ineffective interception—"intercepting" our own striker, which, as we amusingly observed in the simulation, turns out to be surprisingly efficient.

In addition, we noticed a rather funny situation during simulation testing: when the ball had already passed the defender, and the defender, the ball, and the goal were aligned on a straight line, the defender, while trying to recover, accidentally dribbled the ball into their own goal.

To address this issue, we extended our algorithm. Based on the sign of $\overrightarrow{BG} * \overrightarrow{BI}$, we determine whether the ball has already moved past the defender. If the result is negative, it means the ball is behind the defender.

In such cases, instead of moving directly toward the standard intercept point, the defender updates the intercept position by shifting it a fixed distance backward on the ball-goal line, and generates a detour point in the direction perpendicular to the ball-goal line. This detour point is temporarily used as the new destination, allowing the defender to reposition safely.

Once the intercept coordinate becomes positive again, indicating that the ball is once again in front of the defender, the robot will resume its motion toward the updated intercept point.

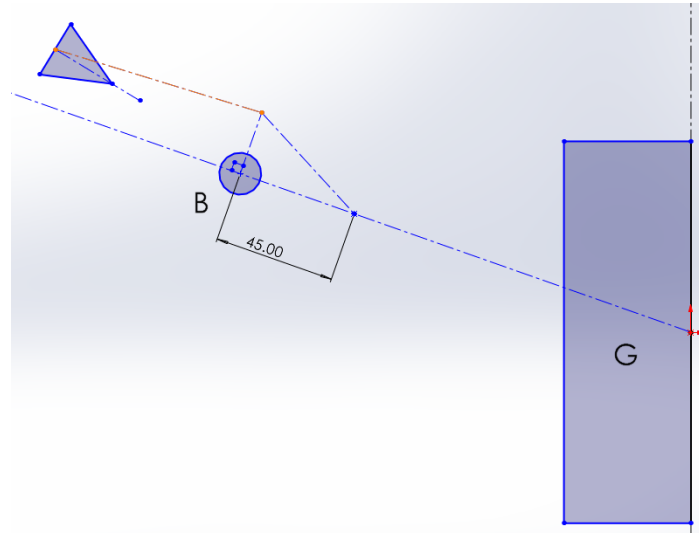


Figure 13. The calculation of intercept point—ball passed defender.

1.5.3 Goalkeeper

In this project, we designed a control function for the goalkeeper robot to continuously face the center line of the field while remaining within the bounds of the penalty area. When the ball enters the Goalkeeper's own half, the robot performs lateral orientation adjustments based on the ball's current position to maintain readiness.

According to the test results, the goalkeeper consistently stayed within a

reasonable defensive range and demonstrated quick and effective adjustment behavior. Given the robot's limited movement speed, it is crucial to initiate orientation changes immediately upon ball entry into the defending half to ensure optimal positioning.

Inspired by videos of real NAO6 robot football matches, we further proposed a ball-saving strategy to address the challenge that the goalkeeper may not have enough time to return to its ideal defensive position. When the system detects that the ball is about to enter the penalty area, a diving save action is triggered.

The goalkeeper robot is able to respond to the ball's trajectory in real-time using sensor data, and both left and right dive motions are executed correctly under the corresponding conditions.

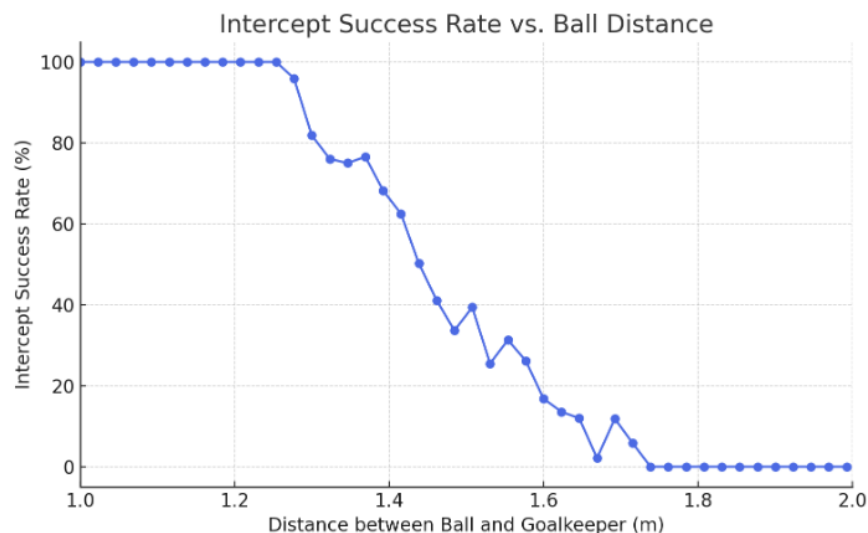


Figure 14. The graph shows the relation between the intercept success and the distance between the ball and the goalkeeper.

In terms of performance, the success rate was 100% when the ball was within 1.26 meters of the robot but dropped to 0% at 1.76 meters. This result highlights the importance of keeping the goalkeeper positioned at the midpoint of the penalty area line, which shortens the response distance and significantly improves the likelihood of a successful save.

1.6 Team Strategy Development and Testing

We designed and tested three different player formation strategies in our simulated robot soccer team:

1.6.1 Strategy 1: Three Hybrid Attackers

Our initial strategy was the simplest version. We divided the entire field into two zones: the front field and the back field, corresponding to the opponent's half and our own half, respectively.

All robots except the goalkeeper would switch between offensive and defensive modes based on the location of the ball. When the ball was in the front field, the robots would actively participate in offense; when the ball entered the back field, they would shift to defensive behaviors.

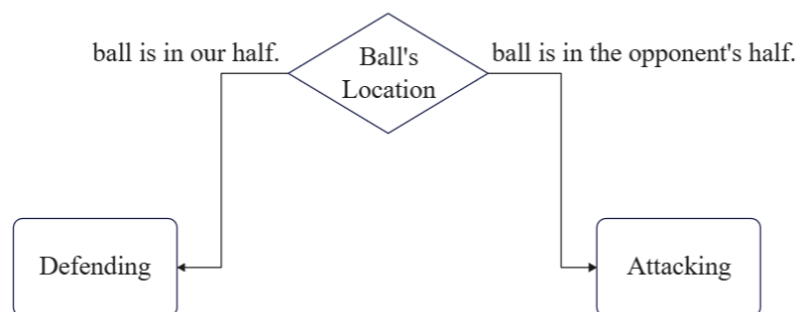


Figure 15. The logic of the hybrid attackers.



Figure 16. The strategy diagram for strategy 1

1.6.2 Strategy 2: One Striker with two Defenders

The second strategy involved dividing the defensive area into left and right halves, with each of the two defenders assigned to guard one half of the field.

At the same time, we applied the interception strategy previously developed specifically for defenders. Each defender would track the ball within their own zone and calculate the optimal interception point to block the opponent's advance effectively.

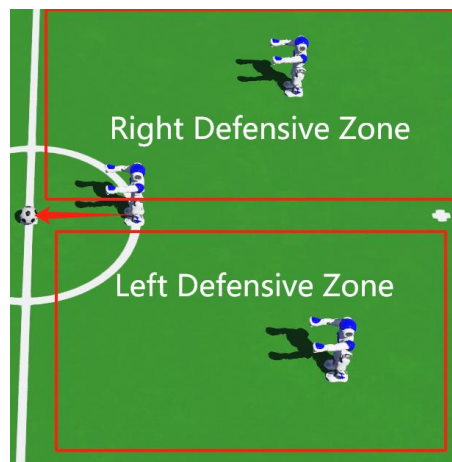


Figure 17. Defensive zone layout diagram.

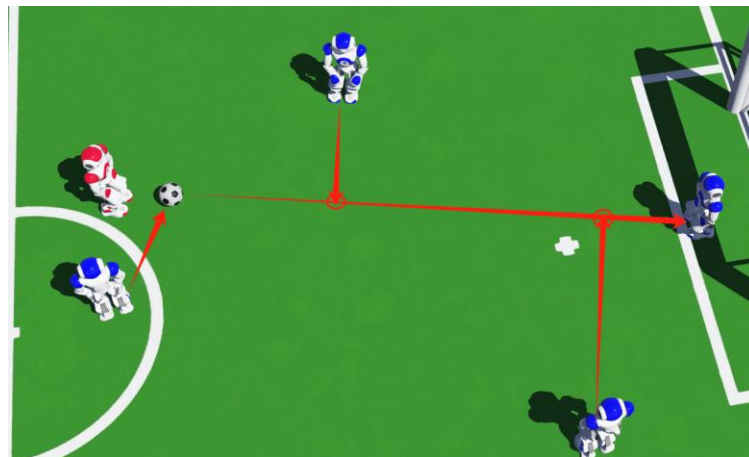


Figure 18. The strategy of interception point.

1.6.3 Strategy 3: One Striker with one Defender and one Flexible Player

In this strategy, one of the defenders is required to dynamically switch between

three roles based on the situation: interceptor, kick to teammate, and kick to opponent.

Conceptually, we divide the field into four quadrants, and the defender's role is determined by evaluating the positions of itself, the teammate defender, the opponent, and the ball.

From an implementation perspective, the quadrant classification is based on comparisons of Manhattan distances, rather than strict geometric partitioning. As a result, the boundaries of each quadrant are not fixed or uniformly sized but are instead determined by relative positional conditions during gameplay. Figure 19. represents the role of switching logic for the multi-function defender based on relative positions and Manhattan distance comparison. The detailed logic is shown in Figure 20.

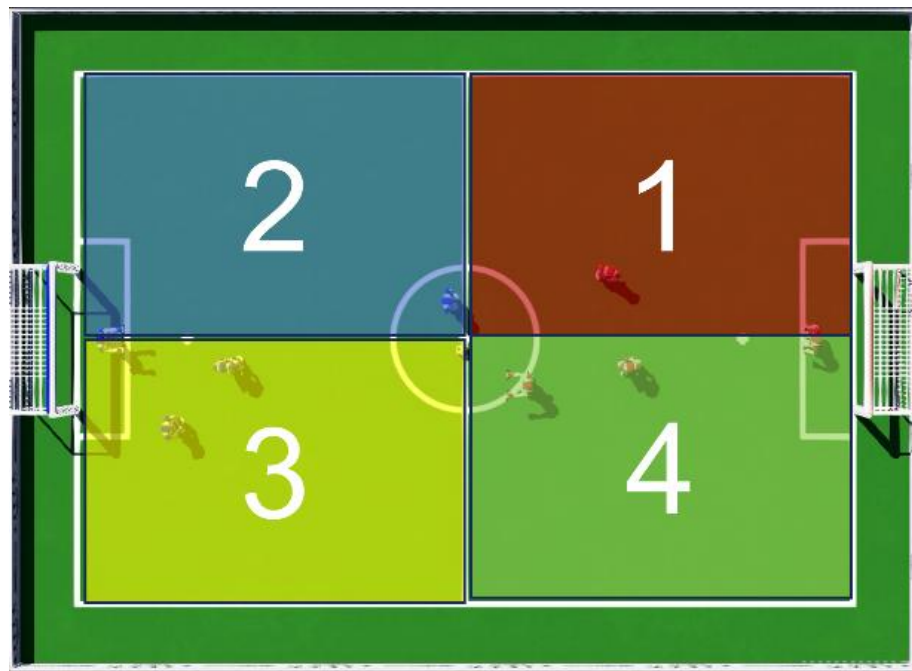


Figure 19. Four quadrants of the field.

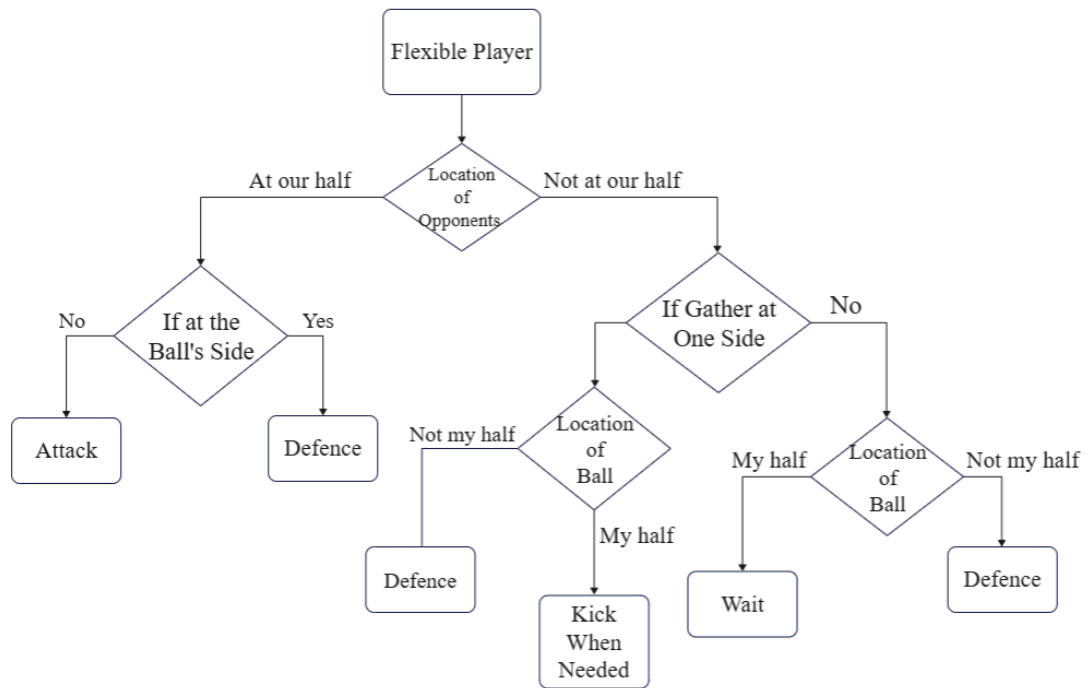


Figure 20. The flow chart for the strategy 3.

2 .Results and Analysis

2.1 PID controller

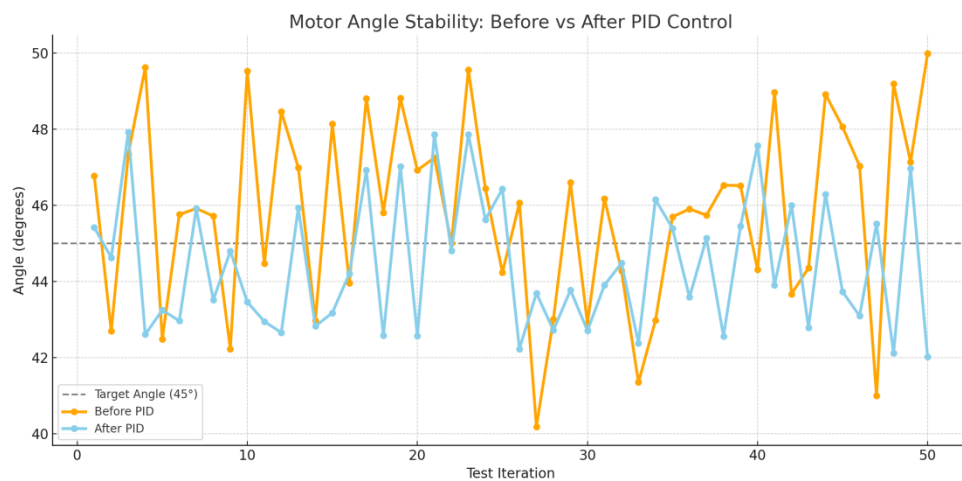


Figure 21. The motor angle stability before and after using the PID controller.

As shown in figure 21, with 45 degrees as the target reference, the orange line represents the motor behavior's result before using the PID controller, and the blue line represents the result after applying the PID controller. The fluctuation in the actual joint angle was significantly reduced after using the PID controller. In the later stages of testing, the robot was no longer prone to falling caused by large deviations in motor positioning.

2.2 Safety Considerations

We integrated the previously developed obstacle avoidance algorithm and self-collision prevention function into one robot and set up another robot without these algorithms as a control group. After running multiple tests, we obtained the results shown in figure 22.

The red bar represents the fall rate before applying the algorithms, while the blue bar represents the fall rate after the algorithms were implemented. As can be clearly observed, the robot that incorporated the algorithms experienced a significant reduction in fall rate. This outcome is highly beneficial in terms of hardware protection and long-term system reliability.

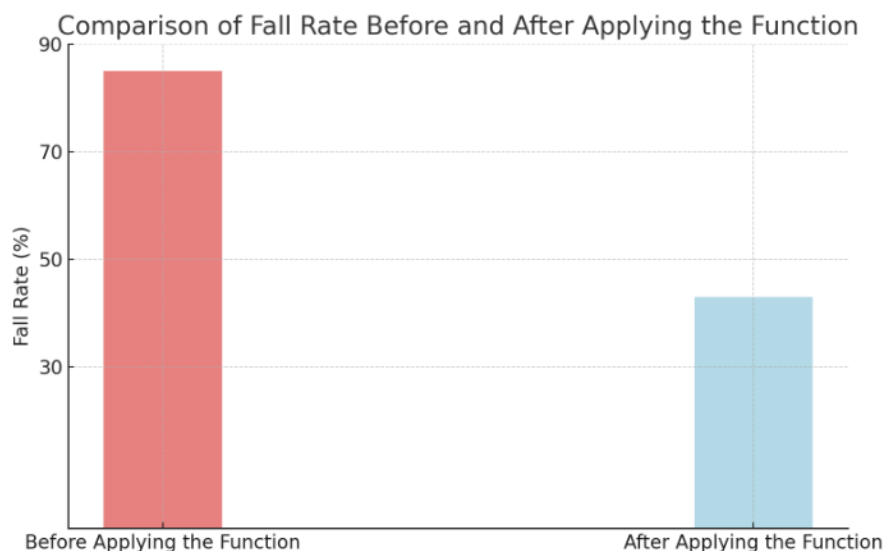


Figure 22. The comparison of the fall rate before and after applying the function.

2.3 Strategy Performance Evaluation

We calculated and compared six key performance indicators under three different strategies: possession rate, shooting rate, scoring rate, fall rate, collision rate, and win rate. The results are shown in figure 23 below.

The yellow boxes represent Strategy 1: *3 Hybrid Attackers*

The orange boxes represent Strategy 2: *1 Striker + 2 Defenders*

The red boxes represent Strategy 3: *1 Striker + 1 Defender + 1 Flexible Player*

As highlighted in the green-framed area, Strategy 3 achieved the highest possession rate, shooting rate, and scoring rate, indicating the strongest offensive efficiency among the three.

In the blue-framed area, both Strategy 2 and Strategy 3 showed significantly lower fall rates and collision rates compared to Strategy 1, suggesting that they offer better hardware protection and safety during operation.

Taking all indicators into account—especially the win rate—we ultimately selected Strategy 3 as the optimal strategy and implemented it in our final robot configuration.



Figure 23. The comparison of key metrics across strategies.

3.Team performance analysis

In figure24, the bar chart illustrates the estimated time and the actual time that we spend in each project stage. In the requirements stage and the algorithm development stage, our team used the exact time that we initially expected. However, the most significant difference is the modeling stage. Our team spent twice the estimated time. It is because we encountered some unexpected difficulties, such as the robot may collide with another or trip over itself. Also, we did not consider some specific situations in the beginning.

Similarly, the operation and test stage took much more time than we anticipated. As we carried out a more comprehensive evaluation of robot behaviors and the collaboration between different characters in simulated environments. The evaluation includes possession rate, number of shots, number of goals, number of falls, and number of collisions. This can help our team to observe the problem and discover our weaknesses of our strategy more easily. Then we can make improvement after identifying the issues. After trying to fix the problems, our team tested the numbers again and repeated them, checking whether they had improved or not.

In addition, the control and guidance stage caused one extra week than we planned. Spending extra time refining the motion stability. Improving the stability of the robots' behaviors, especially in specific situations. These can ensure our robots remain stable in all situations.

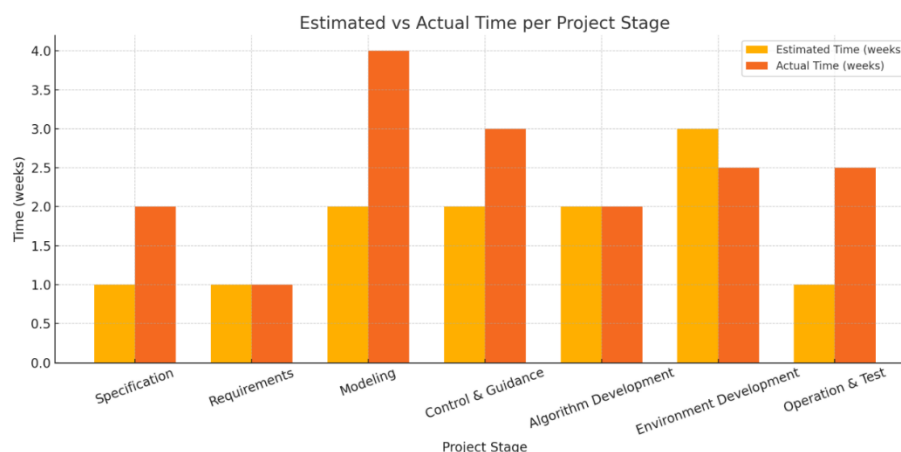


Figure 24. Bar chart showing estimated and actual time for each project stage.

4. Conclusions and Further Work

4.1 Conclusions

4.1.1 Technical Achievements

1. The kinematic model of the robot was successfully established, and a PID controller was integrated for error compensation and regulation, enabling stable robot motion. Building upon this foundation, we further completed the implementation of basic robot actions. We independently developed two light kick motion files, specifically for the left foot and right foot, and also designed and implemented a powerful kick motion and successfully implemented inter-robot communication and cooperative behavior, enabling the robots to share information and work collaboratively in real time.

2. Our team developed a ball tracking algorithm and implemented our own dribbling strategy. At the same time, we addressed the issue from a hardware protection perspective by reducing the robot's fall rate. These improvements greatly enhanced both offensive efficiency and the overall safety of the robot.

3. Our team developed appropriate algorithms for each robot and successfully implemented advanced team strategies. By comparing the performance of three different strategic approaches, we identified and adopted the most effective strategy for our final implementation.

4.1.2 Teamwork & Collaboration.:

1. Held regular weekly meetings to update progress and address issues.
2. Collaborated via GitHub for code integration.
3. Clearly divided tasks based on individual strengths and expertise.
4. Supported each other when facing technical challenges.
5. Maintained close communication to share ideas and feedback.

4.2 Future Work

We would like to express our sincere gratitude to **Dr. McGookin**, **Supervisor Mr. McGir**, and **Supervisor Mr. Wei** for raising critical questions and providing constructive feedback throughout the project.

Based on our current work, we plan to focus on improvements in the following two areas in the future:

1. Transition Zones for Region Boundaries

In the current implementation, we did not consider the potential issues caused when the ball stops near the boundary of a designated region. This may lead to confusion or misjudgment in robot behavior. To address this, we plan to introduce transition zones between regions and develop corresponding algorithms and strategies to handle such boundary conditions more effectively.

2. Integrating PID Control into Light Kick

Currently, our light kick does not incorporate PID control. In the future, we plan to enhance this motion by applying a PID controller, allowing both light kick and powerful kick to be unified under a single behavior called “kick”. We also aim to add decision-making logic, enabling the robot to autonomously choose between kick types based on environmental context, rather than relying on hard-coded manual triggers.

5. Reference

- [1]. C. Sahu, D. R. Parhi, P. B. Kumar, M. K. Muni, A. Chhotray, and K. K. Pandey, "Humanoid NAO: A Kinematic Encounter," *Robotica*, vol. 39, pp. 1997–2007, 2021. doi: 10.1017/S0263574721000096
- [2]. N. Kofinas, E. Orfanoudakis, and M. G. Lagoudakis, "Complete Analytical Inverse Kinematics for NAO," Intelligent Systems Laboratory, Technical University of Crete, Tech. Rep., 2013.
- [3]. Dezong Zhao.2025. course notes 2025: Section 1 Introduction and simple guidance methods . Autonomous Vehicle Guidance Systems ENG:4175 & ENG:5017. James Watt School of Engineering
- [4]. S. A. Nugroho, A. S. Prihatmanto, and A. S. Rohman, "Design and implementation of kinematics model and trajectory planning for NAO humanoid robot in a tic-tac-toe board game," in *2014 IEEE 4th Int. Conf. System Engineering and Technology (ICSET)*, Bandung, Indonesia, Nov. 2014.
- [5]. Cyberbotics Ltd., "*Webots User Guide*," online documentation, Release R2025a. [Online]. Available: <https://cyberbotics.com/doc/guide/index>. Accessed: Oct. 25, 2024.
- [6]. Aldebaran Robotics, "*NAO v6 Technical Dimensions*," online documentation. [Online]. Available: https://doc.aldebaran.com/2-8/family/nao_technical/dimensions_naov6.html. Accessed: Oct. 11, 2025.

6. Appendices

6.1 Python code for light kick

```
def kick_ball(self):
    if self.kick_stage == 0:
        print('Stage 0: Weight shift', flush=True)
        self.motors['LHipRoll'].setPosition(0.3)
        self.motors['RHipRoll'].setPosition(0.2)
        self.motors['LAnkleRoll'].setPosition(-0.2)
        self.motors['RAnkleRoll'].setPosition(-0.2)
        self.kick_stage = 1
        return 10

    elif self.kick_stage == 1:
        print('Stage 1: bend the Left leg', flush=True)
        self.motors['LKneePitch'].setPosition(2.1)
        self.motors['LHipPitch'].setPosition(0.17)
        self.motors['LAnklePitch'].setPosition(-1.0)
        self.motors['RShoulderRoll'].setPosition(-0.26)
        self.motors['LShoulderRoll'].setPosition(-0.26)
        self.kick_stage = 2
        return 10

    elif self.kick_stage == 2:
        print("Stage 2 : kick", flush=True)
        self.motors['LHipPitch'].setPosition(-1.22)
        self.motors['LAnklePitch'].setPosition(-0.3)
        self.kick_stage = 3
        return 10

    elif self.kick_stage == 3:
        self.motors['LKneePitch'].setPosition(1.1)
        self.kick_stage = 4
        return 20

    else:
        print('Kick complete', flush=True)
        for name in self.motor_names:
            if name in self.motors:
                self.motors[name].setPosition(0.0)
        self.is_kicking = False
        self.kick_stage = 0
```

Note: Separating the kick motion into four stages: shifting weight, bending the left leg, kicking, recovering the leg, and resetting the initial position. Setting different joints in each stage.

6.2 Python code for powerful kick

```
target_x = -0.3
target_y = 0.0

error_x = target_x - com_x
error_y = target_y - com_y
total_error = np.sqrt(error_x**2 + error_y**2)
print(f"Total error: {total_error:.3f}", flush=True)

self.pid['integral'] = np.clip(self.pid['integral'] + total_error, -1.0, 1.0)
derivative = total_error - self.pid['prev_error']

control_signal = (
    0.3 * error_x +
    self.pid['kp'] * total_error +
    self.pid['ki'] * self.pid['integral'] +
    self.pid['kd'] * derivative
)
```

Note: `com_x` and `com_y` represent the current position of the center of the mass. For the PID terms, `kp`, `ki`, and `kd` are the proportional, integral, and derivative gains in the PID controller.

6.3 Python code for checking whether to stand up or not

```
def standupIfnecessary(self):
    # Get current acceleration values from the accelerometer
    # Acc = [x, y, z] acceleration in m/s^2
    Acc = self.accelerometer.getValues()

    # Determine if the robot has fallen by analyzing the orientation of the acceleration vector:
    # - A typical upright robot would have Acc[2] close to 9.8 (gravity),
    #   and Acc[0] and Acc[1] near 0.
    # - If Acc[2] is significantly less than 9.8 (e.g., < 5.0), and either Acc[0] or Acc[1] is large
    #   in magnitude (>|4.0|), the robot is likely lying on its side, face, or back.

    if (
        (Acc[2] < 5.0 and Acc[0] < -4.0) or
        (Acc[2] < 5.0 and Acc[0] > 4.0) or
        (Acc[2] < 5.0 and Acc[1] < -4.0) or
        (Acc[2] < 5.0 and Acc[1] > 4.0)
    ):
        # Robot is likely not upright - needs to stand up
        return True
    else:
        # Robot is in normal orientation - no action needed
        return False
```

6.4 Python code for saving ball

```
elif self.hustle_status == HUSTLE.HUSTLE_RIGHT:
    print("Hustle hustle_right!")
    # Execute right-side hustle motion (diving posture)
    self.setMotorPosition("RShoulderRoll", -1.326)
    self.setMotorPosition("RShoulderPitch", -2.08)
    self.setMotorPosition("RElbowYaw", 0.0)
    self.setMotorPosition("RElbowRoll", 0.0)
    # self.setMotorPosition("RWristYaw", 1.82)
    self.setMotorPosition("RHipRoll", 0.379)
    self.setMotorPosition("LHipRoll", 0.379)

    # Wait until all motions complete before proceeding to finish
    if (self.getMoveStage('RShoulderRoll') is move_status.END and
        self.getMoveStage("RShoulderPitch") is move_status.END and
        self.getMoveStage('RElbowYaw') is move_status.END and
        self.getMoveStage("RElbowRoll") is move_status.END and
        self.getMoveStage("LHipRoll") is move_status.END):
        self.hustle_status = HUSTLE.WAITING
    return

elif self.hustle_status == HUSTLE.HUSTLE_LEFT:
    print("Hustle hustle_left!")
    # Execute left-side hustle motion (diving posture)
    self.setMotorPosition("LShoulderRoll", 1.326)
    self.setMotorPosition("LShoulderPitch", 2.08)
    self.setMotorPosition("LElbowYaw", 0.0)
    self.setMotorPosition("LElbowRoll", 0.0)
    self.setMotorPosition("LHipRoll", -0.379)
    self.setMotorPosition("RHipRoll", -0.379)

    # Wait until all motions complete before entering WAITING
    if (self.getMoveStage('LShoulderRoll') is move_status.END and
        self.getMoveStage("LShoulderPitch") is move_status.END and
        self.getMoveStage('LElbowYaw') is move_status.END and
        self.getMoveStage("LElbowRoll") is move_status.END and
        self.getMoveStage("LHipRoll") is move_status.END and
        self.getMoveStage("RHipRoll") is move_status.END):
        self.__temp_time = self.getTime() # Record current time
        self.hustle_status = HUSTLE.WAITING
    return
```

Note: The action of saving the ball is achieved by changing the center of gravity of the robot. Once the center of gravity of the robot is shifted to one side, it can fall in that direction

6.5 Python code for APF algorithm

```
def calculate_navigation_vector(self, defender_pos, intercept_pos, avoid_objects, avoid_radius=1): 5 用法  ZKen-Wong
    """
    repulse_factor = 1.5

    # 计算吸引力 (指向拦截点) attractive force (to intercept point)
    vector_to_intercept = intercept_pos[0:2] - defender_pos[0:2]
    distance_to_intercept = np.linalg.norm(vector_to_intercept)
    force_attract = vector_to_intercept / (distance_to_intercept + 1e-6)

    # 计算斥力 (避让所有障碍物) repulsive force (away from obstacle)
    total_repulse_force = np.array([0.0, 0.0])

    def compute_repulsive_force(obstacle_pos):  ZKen-Wong
        vector_to_obstacle = obstacle_pos[0:2] - defender_pos[0:2]
        distance_to_obstacle = np.linalg.norm(vector_to_obstacle)

        if distance_to_obstacle < avoid_radius:
            # repulsive force = repulse_factor * log(1 + (R - d) / R), avoiding sudden change
            repulse_strength = -repulse_factor * np.log(1 + (avoid_radius - distance_to_obstacle) / avoid_radius)
            force = repulse_strength * (vector_to_obstacle / distance_to_obstacle)
        else:
            force = np.array([0.0, 0.0]) # 超过避让范围不施加力 no repulsive force outside range

        return force

    for obstacle in avoid_objects:
        total_repulse_force += compute_repulsive_force(obstacle[0:2])
    # 合力 = 吸引力 + 总斥力 Total force = attractive + total repulsive
    navigation_vector = force_attract + total_repulse_force
    navigation_vector = navigation_vector / (np.linalg.norm(navigation_vector) + 1e-6) # 归一化 normalized
    # print(f"navigation_vector:{navigation_vector}")
    return navigation_vector
```

Note: This is the specific function of APF algorithm. With inputs of self position, list of obstacles positions, target position, and avoid_radius, function gives back a unit vector. Other navigation functions will use this vector to guide the robot.

6.6 Python code for Recover defensively

```
"""calculate the intercept point, to see if it is too late to intercept"""
"""if impossible,choose the point at the front of the ball to intercept"""
unit_vector_ball2goal = vector_ball2goal/np.abs(np.linalg.norm(vector_ball2goal))

vector_ball2intercept = ori_intercept_position - football_position[0:2]
dot_product = np.dot(unit_vector_ball2goal, vector_ball2intercept)
...

if dot_product <=0.6: #球越过了defender ball has passed defender
    if dot_product <= 0:
        print("dot<0")
        # 计算球的法向量方向(球-球门连线的垂直方向) Direction of the normal vector of the ball-goal line
        perp_vector = np.array([-unit_vector_ball2goal[1], unit_vector_ball2goal[0]])
        # 选取正确的绕行方向(确保不往球门方向绕) Choose correct direction to go around
        if np.dot(perp_vector, robot_position[:2] - football_position[:2]) < 0:
            perp_vector = -perp_vector
        d = 0.4 # 设定绕行距离 d Set detour distance d
        new_intercept_position = football_position[:2] + d * perp_vector # **绕行点范围约束, 避免超出边界**
        new_intercept_position = self.over_range_detect(new_intercept_position) # over field range detect
        # 计算机器人到绕行点的向量
        vector_new = self.calculate_navigation_vector(robot_position, new_intercept_position, avoid_objects)
        angle = self.compute_intercept_angle(vector_new, orientation) # 计算角度 calculate intercept angle
        return angle, np.linalg.norm(new_intercept_position - robot_position[:2])
    else:
        new_intercept_position = self.calculate_equilateral_intercept(football_position, robot_position, goal_position)
        ...
        new_intercept_position = self.over_range_detect(new_intercept_position) # over field range detect
        distance = np.linalg.norm(new_intercept_position - robot_position[0:2])
        ...
        vector_new = self.calculate_navigation_vector(robot_position, new_intercept_position, avoid_objects)
        angle = self.compute_intercept_angle(vector_new, orientation)
        return angle,distance

vector_intercept = self.calculate_navigation_vector(robot_position, intercept_position, avoid_objects)
intercept_angle = self.compute_intercept_angle(vector_intercept,orientation)

return intercept_angle,distance
```

Note: This is the core part of recover defensively for Defender. ‘dot_product’ is $\overrightarrow{BG} * \overrightarrow{BI}$, representing the distance between ball and original intercept point.

6.7 The link of GitHub

Here is our repository of this project: https://github.com/RickyEIE/Nao_test .

Everyone is welcome to visit and please leave your issues.